

# Анализ графов

Пентегова Мария Сергеевна

Группа: 25.Б41-мм

Кафедра: системного программирования

Научный руководитель: Григорьев Семён Вячеславович

Номер семестра практики: 2

## 1. Введение

BFS — один из фундаментальных алгоритмов на графах, применяемый в социальных сетях, веб-поиске и анализе транспортных сетей. С ростом объёмов данных классические реализации BFS на больших графах работают недостаточно быстро, поэтому возникает потребность в высокопроизводительных библиотеках, таких как SuiteSparse:GraphBLAS, основанных на линейной алгебре.

## 2. Постановка задачи

**Цель работы** — сравнить производительность классической (очередь и CSR) и матричной (GraphBLAS) реализаций BFS на графах различных типов: социальных сетях, дорожных сетях, веб-графах и графах цитирования.

**Задачи:**

1. реализовать классический и матричный BFS (один и несколько источников);
2. реализовать чтение `.mtx`, преобразование в CSR, поиск вершины с максимальной степенью, замер времени;
3. провести эксперименты на реальных графах из коллекции SuiteSparse;
4. проанализировать результаты и выявить закономерности.

## 3. Обзор

В литературе описаны отдельные примеры ускорения BFS через GraphBLAS, однако систематического сравнения классической и матричной реализаций на графах разных типов — как ориентированных, так и неориентированных — с учётом ширины фронтов BFS не проводилось. Остаётся неясным, в каких случаях быстрее GraphBLAS, а в каких классический BFS.

## 4. Описание предлагаемого решения

Решение — программа на C из четырёх компонентов:

- **Модуль работы с графом** (`utils.c`, `utils.h`) — чтение графа из `.mtx`, хранение списков смежности, преобразование в CSR, поиск вершины с максимальной степенью.
- **Классический BFS** (`classic_bfs.c`) — обход на очереди и CSR (один и несколько источников).

- **GraphBLAS BFS** (`graphblas_bfs.c`) — обход через матрично-векторное умножение (один и несколько источников).
- **Точка входа и бенчмарк** (`main.c`, `benchmark.c`) — запуск всех четырёх реализаций, замер времени, вывод результатов.

## 4.1. Используемые технологии

- **Язык:** C11.
- **Библиотека:** SuiteSparse:GraphBLAS (сборка из исходников).
- **Система сборки:** CMake 3.10+.
- **Измерение времени:** `clock_gettime()` с `CLOCK_MONOTONIC`.
- **Тестирование:** CTest.

## 4.2. Формат хранения графа

Выбран **CSR (Compressed Sparse Row)**: компактное хранение разреженных графов за счёт массивов `row_ptr` и `col_idx`, быстрый доступ к соседям за  $O(\deg(v))$ , прямой импорт в GraphBLAS.

# 5. Эксперименты

## 5.1. Цель экспериментов

Проверить, что производительность BFS зависит не только от размера графа, но и от его топологии, и определить, когда быстрее классика, а когда GraphBLAS.

## 5.2. Методика проведения

Эксперименты проводились на 7 реальных графах из коллекции SuiteSparse. Для каждого графа выбиралась стартовая вершина с максимальной степенью (один источник) и четыре равномерно распределённые вершины (несколько источников).

Каждый алгоритм запускался **10 раз**; фиксировались среднее (Avg), минимальное (Min) и максимальное (Max) время в микросекундах (`clock_gettime()`, `CLOCK_MONOTONIC`). Перед замерах выполнялся один дополнительный запуск для исключения разовых издержек инициализации потоков GraphBLAS.

## 5.3. Результаты

Сводная таблица средних значений по 10 запускам (время в микросекундах):

| Граф             | Classic Parent | Classic Multi | GB Level   | GB Multi   | Ускорение       |
|------------------|----------------|---------------|------------|------------|-----------------|
| com-Orkut        | 2 652 628      | 2 700 768     | 486 500    | 420 307    | 5.45x (GB)      |
| soc-LiveJournal1 | 1 915 627      | 1 976 468     | 358 255    | 331 458    | 5.35x (GB)      |
| road_usa         | 1 451 387      | 1 588 466     | 25 839 879 | 22 121 848 | 0.06x (Classic) |
| roadNet-CA       | 127 543        | 110 558       | 663 630    | 670 218    | 0.19x (Classic) |
| web-Google       | 100 233        | 80 683        | 101 149    | 106 972    | 0.99x (равно)   |
| cit-Patents      | 17 500         | 4 730         | 181 639    | 46 757     | 0.10x (Classic) |
| com-Amazon       | 38 967         | 31 974        | 61 923     | 62 144     | 0.63x (Classic) |

Таблица 1: Средние значения времени выполнения (мкс) по 10 запускам

## 5.4. Анализ результатов

Универсального решения нет — выбор алгоритма зависит от трёх факторов в порядке иерархии:

1. **Ширина фронтов BFS** — главный фактор: широкие фронты (социальные сети) → GraphBLAS; узкие (дорожные сети, графы цитирования) → классика.
2. **Размер графа** (число рёбер): огромные графы (> 50М рёбер) → GraphBLAS при широких фронтах; маленькие и средние (< 5М рёбер) → классика из-за накладных расходов GraphBLAS.
3. **Топология** — социальные сети дают широкие фронты, дорожные сети и графы цитирования — узкие.

Отдельно стоит отметить граф `cit-Patents`: он ориентированный, но классика выиграла с большим отрывом. Значит, ориентированность сама по себе не гарантирует победу GraphBLAS.

## 5.5. Тестирование корректности

Тесты (`assert.h`, `CTest`) проверяют обе реализации на графе из 5 вершин.

- **Classic BFS**: все вершины посещены, корень указывает на себя, у не-корневых нет петель, родители совпадают с ожидаемыми для стартов 0 и 1; в `multisource` все источники указывают на себя.
- **GraphBLAS BFS**: все вершины посещены, расстояние до старта равно 0, уровни совпадают с ожидаемыми, каждая вершина имеет соседа на уровень выше; в `multisource` расстояния не больше, чем при обходе от одного источника.
- **Сравнительные тесты**: результаты классики и GraphBLAS совпадают поэлементно.

Все тесты проходят (100% passed), что подтверждает корректность обеих реализаций и их эквивалентность.

## 6. Заключение

- Производительность BFS зависит от топологии графа и ширины фронтов, а не только от размера.
- GraphBLAS максимально эффективен на больших ориентированных графах с широкими фронтами.
- Классический BFS быстрее на графах с узкими фронтами и на маленьких графах.
- Ориентированность не гарантирует эффективность GraphBLAS

Репозиторий: [https://github.com/MariaPentegova/graph\\_blas\\_bfs-2/](https://github.com/MariaPentegova/graph_blas_bfs-2/)

Техническая документация: [https://github.com/MariaPentegova/graph\\_blas\\_bfs-2/blob/main/README.md](https://github.com/MariaPentegova/graph_blas_bfs-2/blob/main/README.md)